

Trabalho Laboratorial 3

Relatório da Unidade Curricular de Automação Inteligente e a Indústria 4.0
Mestrado em Engenharia Eletrotécnica
2024/2025

Autores:

	
Nome: João Paulino Nº de Estudante: 2230078	Nome: Judá Imbó Nº de Estudante: 2232626

Índice

Índice de Figuras	IV
1. Introdução	5
2. Máquina de Injeção e Servidor	6
2.1. Máquina de Injeção	6
2.2. Servidor da máquina de injeção	8
3. Sistema de Extração de Peças	11
4. MES	18
5. IMM Energy Monitor e X Monitor	23
6. Conclusão	25

Índice de Figuras

Figura 1 – Descrição da interligação entre a máquina de injeção e os sistemas externos.....	5
Figura 2 – Variáveis declaradas no TIA Portal.....	7
Figura 3 – Function Block “MB_SERVER”.....	7
Figura 4 – Ativação e execução da etapa 2 no TIA Portal.....	8
Figura 5 – Ativação da etapa inicial no TIA Portal.....	8
Figura 6 – Definição do endereço IP e da porta TCP, para ligação ao servidor Modbus	8
Figura 7 – Declaração das variáveis no servidor OPC-UA.....	9
Figura 8 – Leitura da variável “etapa” no servidor OPC-UA	10
Figura 9 – Exemplo de escrita da variável “extraction_end”	10
Figura 10 - Interface gráfica Extraction Client	11
Figura 11 - Endereço dos servidores	12
Figura 12 - Declaração de variáveis.....	12
Figura 13 - Subscrição da variável “AtoB”.....	13
Figura 14 - Subscrição da variável "start_extraction"	14
Figura 15 - Lógica da função “monitoredItem_Notification” para “AtoB”	15
Figura 16 - Função "FimExtracaoPeca_ON()"	15
Figura 17 - Lógica da função “monitoredItem_Notification” para “start_extraction”	16
Figura 18 - Função "Inicio_Extracao_Peca()"	16
Figura 19 - Função "FimExtracaoPeca_OFF()"	17
Figura 20 - Interface gráfica MES Client.....	18
Figura 21 - Endereço do servidor.....	19
Figura 22 - Declaração de variáveis.....	19
Figura 23 - Subscrição da variável “etapa”	20
Figura 24 – Condição para apresentar valor de “etapa”	20
Figura 25 - Lógica da função “button1_Click”	21
Figura 26 - Função "nserie_Changed()"	22
Figura 27 – Cálculos da tensão e da corrente	23
Figura 28- Esquema de acesso aos dados do servidor e de criação da dashboard	24
Figura 29 - Visualização e atualização dos valores da corrente e da tensão em tempo real	24

1. Introdução

Este relatório, referente ao trabalho laboratorial 3 da Unidade curricular de Automação Inteligente e a Indústria 4.0, tem como objetivo apresentar a exploração dos conceitos práticos e das tecnologias associadas à automação avançada e a Indústria 4.0. O foco principal está na interligação e comunicação entre uma máquina de injeção (considerada neste caso de estudo como um equipamento *legacy*) e sistemas externos como a monitorização energética e a extração da peça por um robô. Para o desenvolvimento deste trabalho, foram utilizados softwares conhecidos na indústria como o TIA Portal, Visual Studio, Node-RED, RobotStudio e PLCnext, de maneira a interligar estes sistemas.

O relatório se encontra organizado em vários capítulos e subcapítulos que descrevem os diferentes sistemas desenvolvidos. O capítulo 2 descreve o servidor da máquina de injeção, enquanto o capítulo 3 aborda o sistema de extração da peça. O capítulo 4 trata da integração de um sistema MES (*Manufacturing Execution System*) e, por fim, a descrição do sistema de monitorização do consumo energético da máquina é focada no capítulo 5.

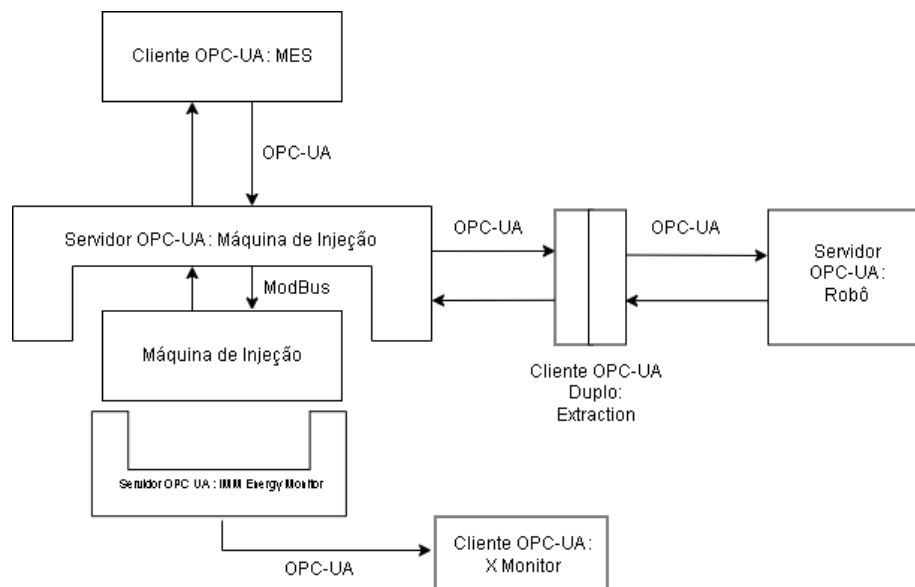


Figura 1 – Descrição da interligação entre a máquina de injeção e os sistemas externos

2. Máquina de Injeção e Servidor

A máquina de injeção é descrita neste trabalho é considerada como um equipamento *legacy* em termos de características da Indústria 4.0. Para tal, foi necessário a criação de um servidor OPC-UA que permita não só a comunicação com a máquina através do protocolo de comunicação Modbus-TCP, como também permita a comunicação com sistemas externos, que serão descritos ao longo do relatório.

2.1. Máquina de Injeção

O processo efetuado pela máquina de injeção é controlado por um autômato Siemens. Este processo é composto por 8 etapas:

0. Máquina pronta (etapa inicial do processo);
1. Alimentação de plástico;
2. Fecho do molde;
3. Injeção;
4. Compactação;
5. Arrefecimento;
6. Abertura do molde;
7. Extração da peça.

No TIA Portal, foram criadas variáveis com as etapas descritas, juntamente com outras específicas destinadas a iniciar o processo (“start”), e a monitorizar a etapa atual do processo (“etapa”). Estas últimas, juntamente com as variáveis relacionadas com a extração da peça, são de interesse comum com o servidor, uma vez que a máquina de injeção compartilhará essas informações diretamente com o servidor.

F1_alim_plastic	Bool	%M1.0
F2_close_mould	Bool	%M1.1
F3_Injection	Bool	%M1.2
F4_Compact	Bool	%M1.3
F5_Arref_mould	Bool	%M1.4
F6_Open_mould	Bool	%M1.5
F7_Extract	Bool	%M1.6
F0_machine_ready	Bool	%M2.0
start	Int	%MW1000
etapa	Int	%MW1002
Start_Extraction	Int	%MW1004
Extraction_End	Int	%MW1006

Figura 2 – Variáveis declaradas no TIA Portal

A comunicação entre o servidor e a máquina de injeção controlada pelo autômato é controlada através do protocolo de comunicação Modbus, Para tal, é usado o *Function Block* correspondente ao servidor Modbus, intitulado “MB_SERVER”. Este permite a troca de dados e a informação dos mesmos em memórias reservadas a iniciar a partir do endereço M1000.0.

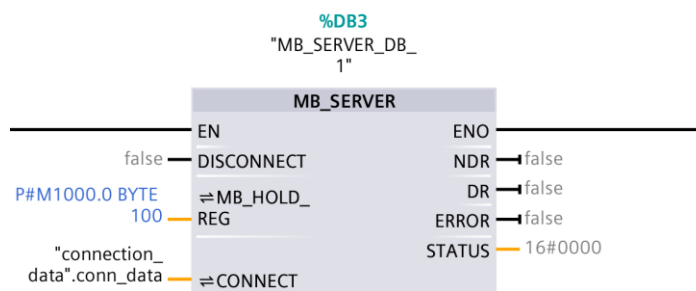


Figura 3 – Function Block “MB_SERVER”

O processo de injeção no TIA Portal é controlado com o uso de temporizadores, com exceção da etapa inicial. Esta etapa requer que nenhuma etapa esteja ativa ao iniciar o processo (ou seja ao ligar a máquina de injeção). Além disso, é necessário que a etapa de extração esteja concluída, sendo esta dependente da resposta do servidor do robô, conforme detalhado mais adiante no relatório.

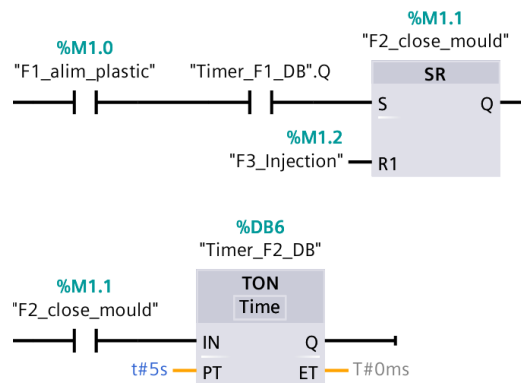


Figura 4 – Ativação e execução da etapa 2 no TIA Portal

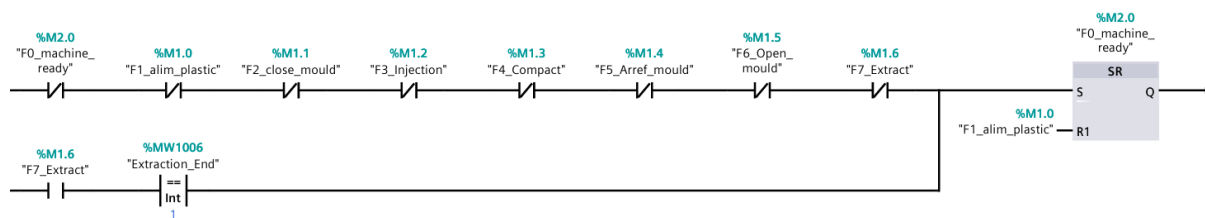


Figura 5 – Ativação da etapa inicial no TIA Portal

2.2. Servidor da máquina de injeção

O servidor OPC-UA da máquina de injeção (com o nome de “MyModbus2Opcua”) permite a comunicação entre a máquina e outros elementos envolvidos no processo, como o robô responsável pela extração da peça e o MES que permite o controlo e a monitorização do processo de injeção. Para tal, é necessário que o servidor se conecte com a máquina de injeção por Modbus. Para atender esta necessidade, foi utilizada a biblioteca Easymodbus configurando o servidor OPC-UA com o endereço IP do autómato da máquina e a porta de comunicação correspondente nos *Private Fields* do servidor, tal como mostra a Figura 5.

```
//IP address of the Modbus server (i.e., IP address of the PLC)
private string Modbus_IP = "192.168.0.1";
//TCP port of the Modbus server (default: 502)
private int Modbus_port = 502;
private ModbusClient Modbus_client;
#endregion
```

Figura 6 – Definição do endereço IP e da porta TCP, para ligação ao servidor Modbus

Assim como a configuração da comunicação entre o servidor OPC-UA da máquina e a mesma, a declaração e o processamento de dados das variáveis foram implementados no ficheiro “MyDiServerNodeManager.cs” do servidor. Essas variáveis estão organizadas em um *array* denominado “m_dynamicNodes”, que facilita o acesso aos dados. A tabela seguinte mostra a indexação do array juntamente com o nome das variáveis no servidor, os tipos de dados declarados e a que endereços correspondem no lado da máquina (TIA Portal) e no servidor Modbus.

Indexação do array m_dynamicNodes	Nome da variável no servidor OPC- UA	Tipo de variável declarada no servidor OPC- UA	Endereço no TIA Portal	Endereço Modbus
0	n_serie	String	-----	-----
1	etapa	Int32	MW1002	1
2	start	Int32	MW1000	0
7	extraction_end	Int32	MW1006	3
8	start_extraction	Int32	MW1004	2

```
//-----MES Folder
//-----
//-----

//create myDevice1 as a simple folder with a variable
FolderState mes_Folder = CreateFolder(DeviceSetObj, "mes_Folder", "mes_Folder");

//create n_serie
BaseDataVariableState n_serie = CreateVariable(mes_Folder, "n_serie", "n_serie", BuiltInType.String, ValueRanks.Scalar);

//initialize var
n_serie.Value = " ";

//store variable in my variable list for latter access (not necessary if variable is declare as a field of the class)
m_dynamicNodes.Add(n_serie);

//add to Address Space
AddPredefinedNode(SystemContext, mes_Folder);

//-----

//create etapa
BaseDataVariableState etapa = CreateVariable(mes_Folder, "etapa", "etapa", BuiltInType.Int32, ValueRanks.Scalar);

//initialize var
etapa.Value = 0;

//store variable in my variable list for latter access (not necessary if variable is declare as a field of the class)
m_dynamicNodes.Add(etapa);

//add to Address Space
AddPredefinedNode(SystemContext, mes_Folder);
```

Figura 7 – Declaração das variáveis no servidor OPC-UA

O processamento de dados no servidor OPC-UA da máquina de injeção é realizado através da função “myDoProcessing” que permite visualizar o que acontece dentro deste servidor. Entre as variáveis processadas, a “etapa” é a única cuja informação é vinda da máquina. Para possibilitar a leitura desta variável, é utilizada a função “ReadHoldingRegisters” da biblioteca EasyModbus (responsável pelo acesso dos dados armazenados no servidor Modbus) e a função “siemens_register2short”, que faz a conversão dos dados recebidos do Modbus para que sejam utilizados no servido OPC-UA, conforme mostra a Figura 7.

```
// READ: 'etapa' (MW1002, Holding Register)
m_dynamicNodes[1].Value = (double)siemens_register2short(Modbus_client.ReadHoldingRegisters(1, 1));
m_dynamicNodes[1].ClearChangeMasks(SystemContext, false);
```

Figura 8 – Leitura da variável “etapa” no servidor OPC-UA

O resto das variáveis declaradas no servidor OPC-UA da máquina de injeção, com exceção da variável “n_serie” (que regista o número de série da peça produzida na máquina), são escritas e enviadas para o TIA Portal por Modbus. A Figura 8 ilustra um exemplo de escrita da variável “extraction_end”, tanto no servidor OPC-UA da máquina como no servidor Modbus, que realiza o envio dos dados para o TIA Portal. O valor “1” será escrito apenas se as variáveis do array “m_dynamicNodes” correspondentes à “etapa” e ao fim de extração da peça (“extraction_end”), apresentarem em simultâneo os valores “7” e “1”. Caso contrário, será registado o valor “0” tanto no servidor Modbus, como no servidor OPC-UA da máquina de injeção.

```
//Código para enviar o sinal extraction_end para o autómato

//se etapa == 7 e extraction_end == 1 :
if (((double)m_dynamicNodes[1].Value == (double)7) && ((Int32)m_dynamicNodes[7].Value == (Int32)1))
{
    //extraction_end = true;

    int[] writeval2 = { 1 };
    Modbus_client.WriteMultipleRegisters(3, writeval2);
}
else
{
    //extraction_end = false;
    m_dynamicNodes[7].Value = (Int32)0;
    int[] writeval2 = { 0 };
    Modbus_client.WriteMultipleRegisters(3, writeval2);
}
m_dynamicNodes[7].ClearChangeMasks(SystemContext, false);
```

Figura 9 – Exemplo de escrita da variável “extraction_end”

3. Sistema de Extração de Peças

Para fazer a extração das peças na última etapa do processo de injeção foi instalado um robô da marca ABB. O robô é um sistema independente que pode ser controlado através do servidor OPC-UA “Robot”, que é simulado através do *software* RobotStudio.

Através da criação do cliente OPC-UA “Extraction” é possível garantir a interligação entre o servidor “IMM Control” e o servidor “Robot”. Este cliente apresenta uma ligação dupla.

Na Figura 10 é possível visualizar o aspeto da interface gráfica do *Extraction Client*.

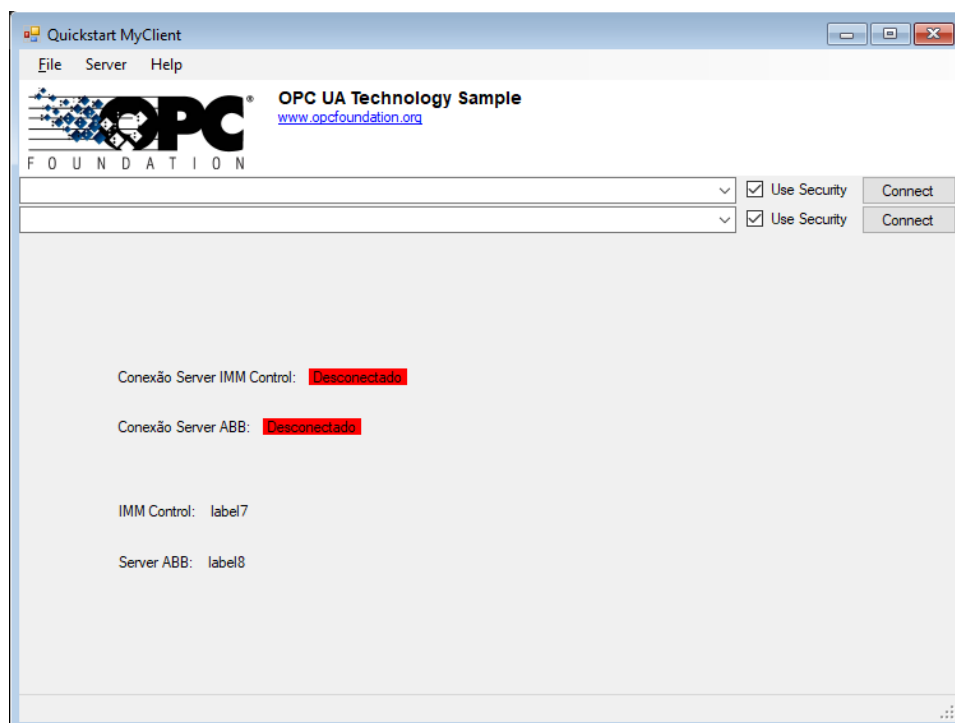


Figura 10 - Interface gráfica *Extraction Client*

Para configuração deste cliente, foi disponibilizada a localização dos servidores a conectar associados ao “ConnectServerCTRL” e “connectServerCtrl2” como pode ser visto na Figura 11 .

```

public MainForm(ApplicationConfiguration configuration)
{
    InitializeComponent();
    ConnectServerCTRL.Configuration = m_configuration = configuration;
    ConnectServerCTRL.ServerUrl = "opc.tcp://localhost:62541/Quickstarts/ServerABB";

    connectServerCtrl2.Configuration = m_configuration = configuration;
    connectServerCtrl2.ServerUrl = "opc.tcp://localhost:62543/Quickstarts/MyModbus20pcuaServer";

    this.Text = m_configuration.ApplicationName;
}

```

Figura 11 - Endereço dos servidores

```

#region Private Fields
private ApplicationConfiguration m_configuration;
private Session m_session;
private bool m_connectedOnce;
private Subscription m_subscription;

private Session m_session2;
private bool m_connectedOnce2;
private Subscription m_subscription2;

//Variáveis Robot ABB
private bool AtoB;
private uint AtoB_handle;

//Variáveis IMM_Control Server
private double start_extraction;
private uint start_extraction_handle;

```

Figura 12 - Declaração de variáveis

Nos *Private Fields*, foi necessário declarar uma cópia das variáveis “m_session”, “m_connectedOnce”, “m_subscription”, para a segunda conexão, uma vez que a comunicação é com dois servidores. Também foram declaradas as respectivas variáveis a serem utilizadas. Na Figura 12 é possível visualizar estas declarações.

Foram realizadas as subscrições das variáveis “AtoB” no nó 2 do “ServerABB” e da subscrição da variável “start_extraction” presente no nó 3 do servidor “IMMControl”. Foi ainda acrescentado um código para ser possível visualizar na interface se a conexão aos servidores foi bem sucedida. Na Figura 13 e Figura 14 é possível visualizar estas subscrições.

```

//----- Server_ConnectComplete -----
1 reference
private void Server_ConnectComplete(object sender, EventArgs e)
{
    try
    {
        m_session = ConnectServerCTRL.Session;

        // set a suitable initial state.
        if (m_session != null && !m_connectedOnce)
        {
            m_connectedOnce = true;
            label3.Text = "Conectado";
            label3.BackColor = Color.PaleGreen;
        }

        //verify if Session is established
        if (m_session != null)
        {
            //verify if the Subscription was not already created
            if (m_subscription == null)
            {
                //create the subscription
                m_subscription = new Subscription(m_session.DefaultSubscription);
                m_subscription.PublishingEnabled = true;
                m_subscription.PublishingInterval = 4000; //msec interval
                m_session.AddSubscription(m_subscription);
                m_subscription.Create();

                MonitoredItem monitoredItem;
                //Add AtoB to monitored items
                monitoredItem = new MonitoredItem(m_subscription.DefaultItem);
                monitoredItem.StartNodeId = "ns=2;s=AtoB";
                monitoredItem.AttributeId = Attributes.Value;
                monitoredItem.MonitoringMode = MonitoringMode.Reporting;
                monitoredItem.SamplingInterval = 1000;
                monitoredItem.QueueSize = 0;
                monitoredItem.DiscardOldest = true;
                //get handle number
                AtoB_handle = monitoredItem.ClientHandle;
                //define event handler for this item, and then add to subscription
                monitoredItem.Notification += new MonitoredItemNotificationEventHandler(monitoredItem_Notification);
                m_subscription.AddItem(monitoredItem);
                m_subscription.ApplyChanges();
            }
        }

        // browse the instances in the server.
        //BrowseCTRL.Initialize(m_session, ObjectIds.ObjectsFolder, ReferenceTypeIds.Organizes, ReferenceTypeIds.Aggregates);
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException(this.Text, exception);
    }
}

```

Figura 13 - Subscrição da variável “AtoB”

```

//----- Server_ConnectComplete2 -----
1 reference
private void Server_ConnectComplete2(object sender, EventArgs e)
{
    try
    {
        m_session2 = connectServerCtrl2.Session;

        // set a suitable initial state.
        if (m_session2 != null && !m_connectedOnce2)
        {
            m_connectedOnce2 = true;
            label4.Text = "Conectado";
            label4.BackColor = Color.PaleGreen;
        }

        //verify if Session is established
        if (m_session2 != null)
        {
            //verify if the Subscription was not already created
            if (m_subscription2 == null)
            {
                //create the subscription
                m_subscription2 = new Subscription(m_session2.DefaultSubscription);
                m_subscription2.PublishingEnabled = true;
                m_subscription2.PublishingInterval = 1000; //msec interval antiga versao: 4000
                m_session2.AddSubscription(m_subscription2);
                m_subscription2.Create();

                MonitoredItem monitoredItem;
                //Add start_extraction to monitored items
                monitoredItem = new MonitoredItem(m_subscription2.DefaultItem);
                monitoredItem.StartNodeId = "ns=3;s=start_extraction";
                monitoredItem.AttributeId = Attributes.Value;
                monitoredItem.MonitoringMode = MonitoringMode.Reporting;
                monitoredItem.SamplingInterval = 1000;
                monitoredItem.QueueSize = 0;
                monitoredItem.DiscardOldest = true;
                //get handle number
                start_extraction_handle = monitoredItem.ClientHandle;
                //define event handler for this item, and then add to subscription
                monitoredItem.Notification += new MonitoredItemNotificationEventHandler(monitoredItem_Notification);
                m_subscription2.AddItem(monitoredItem);
                m_subscription2.ApplyChanges();
            }
        }

        // browse the instances in the server.
        //BrowseCTRL.Initialize(m_session2, ObjectIds.ObjectsFolder, ReferenceTypeIds.Organizes, ReferenceTypeIds.Aggregates);
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException(this.Text, exception);
    }
}

```

Figura 14 - Subscrição da variável "start_extraction"

Na função “monitoredItem_Notification” foi criada a lógica relativa à extração da peça por parte do robô ABB. Caso a variável “AtoB” seja ativa é escrito o texto “A Extrair Peça” a vermelho, caso contrário é porque a peça já foi extraída, e nesse caso escreve “Peça Extraída” a cinzento e realiza a função “FimExtracaoPeca_ON()” que coloca o valor 1 na variável “extraction_end” do servidor “IMMControl”. Na Figura 15 é apresentada a lógica presente na função “monitoredItem_Notification”, e na Figura 16 a função “FimExtracaoPecaON()”.

```

//----- Server Robot ABB -----
if (monitoredItem.ClientHandle == AtoB_handle)
{
    AtoB = (Boolean)notification.Value.WrappedValue.Value;

    if (AtoB)
    {
        //Se o Robo estiver na extração da peça irá alterar a informação do MainForm Design (Cliente)
        // e coloca uma variável de controle a TRUE
        label8.BackColor = Color.Red;
        label8.Text = "A Extrair Peça";
    }
    else
    {
        //Caso o Robo esteja parado a informação da MainForm Design (Cliente)
        //Caso já tenha extraído uma peça é chamada a Função (Fim_Extracao_Peca)
        label8.BackColor = Color.Gray;
        label8.Text = "Peça Extraída - A Aguardar Extração";
        FimExtracaoPeca_ON();
    }
}
}

```

Figura 15 - Lógica da função “monitoredItem_Notification” para “AtoB”

```

//----- Função que permite enviar informação para o Servidor IMM_Control para dizer que terminou de Extrair a Peça -----
1 reference
private void FimExtracaoPeca_ON()
{
    try
    {
        if (m_session2 != null)
        {
            Int32 extraction_end = 1;
            WriteValue valueToWrite = new WriteValue();
            valueToWrite.NodeId = "ns=3;s=extraction_end";
            valueToWrite.AttributeId = Attributes.Value;
            valueToWrite.Value.Value = extraction_end;
            valueToWrite.Value.StatusCode = StatusCodes.Good;
            valueToWrite.Value.ServerTimestamp = DateTime.MinValue;
            valueToWrite.Value.SourceTimestamp = DateTime.MinValue;
            WriteValueCollection valuesToWrite = new WriteValueCollection();
            valuesToWrite.Add(valueToWrite);
            // write current value.
            StatusCodeCollection results = null;
            DiagnosticInfoCollection diagnosticInfos = null;
            m_session2.Write(
                null,
                valuesToWrite,
                out results,
                out diagnosticInfos);
            ClientBase.ValidateResponse(results, valuesToWrite);
            ClientBase.ValidateDiagnosticInfos(diagnosticInfos, valuesToWrite);
            if (StatusCode.IsBad(results[0]))
            {
                throw new ServiceResultException(results[0]);
            }
        }
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException("Error Writing Value", exception);
    }
}

```

Figura 16 - Função “FimExtracaoPeca_ON()”

Na função “monitoredItem_Notification” foi criada também a lógica relativa ao server IMM Control para que caso a variável “start_extraction” apresente o valor lógico 1, o mesmo escreve “Extração de Peça” e realiza a função “Inicio_Extracao_Peca()” que coloca o valor *true* na variável “AtoB” do servidor Robot. Caso contrário, é porque se encontra em espera e desse modo escreve “Em Espera” e realiza a função “FimExtracaoPeca_OFF()” que coloca o valor 0 na variável “extraction_end” do servidor IMM. Na Figura 17 é apresentada a lógica presente na função “monitoredItem_Notification”, na Figura 18 a função “Inicio_Extracao_Peca()” e na Figura 19 a função “FimExtracaoPeca_OFF()”.

```

//----- Server IMM_Control -----
//Server IMM_Control
if (monitoredItem.ClientHandle == start_extraction_handle)
{
    start_extraction = (double)notification.Value.WrappedValue.Value;

    if (start_extraction == (Int32)1)
    {
        //Se Existir ordem de extração por parte do Robot ABB
        label7.BackColor = Color.Green;
        label7.Text = "Extração de Peça";
        Inicio_Extracao_Peca();
    }
    else
    {
        //Caso contrario envia informacao de que a peça ainda nao foi extraida
        label7.BackColor = Color.Red;
        label7.Text = "Em Espera";
        FimExtracaoPeca_OFF();
    }
}

```

Figura 17 - Lógica da função "monitoredItem_Notification" para "start_extraction"

```

//--- Função que permite enviar informação para o Servidor do Robot ABB para dar ordem para Iniciar Extração ---
// reference
private void Inicio_Extracao_Peca()
{
    try
    {
        if (m_session != null)
        {
            Boolean ordem = true;
            WriteValue valueToWrite = new WriteValue();
            valueToWrite.NodeId = "ns=2;s=AtoB";
            valueToWrite.AttributeId = Attributes.Value;
            valueToWrite.Value.Value = ordem;
            valueToWrite.Value.StatusCode = StatusCodes.Good;
            valueToWrite.Value.ServerTimestamp = DateTime.MinValue;
            valueToWrite.Value.SourceTimestamp = DateTime.MinValue;
            WriteValueCollection valuesToWrite = new WriteValueCollection();
            valuesToWrite.Add(valueToWrite);
            // write current value.
            StatusCodeCollection results = null;
            DiagnosticInfoCollection diagnosticInfos = null;
            m_session.Write(
                null,
                valuesToWrite,
                out results,
                out diagnosticInfos);
            ClientBase.ValidateResponse(results, valuesToWrite);
            ClientBase.ValidateDiagnosticInfos(diagnosticInfos, valuesToWrite);
            if (StatusCode.IsBad(results[0]))
            {
                throw new ServiceResultException(results[0]);
            }
        }
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException("Error Writing Value", exception);
    }
}

```

Figura 18 - Função "Inicio_Extracao_Peca()"


```

1 reference
private void FimExtracaoPeca_OFF()
{
    try
    {
        if (m_session2 != null)
        {
            Int32 extraction_end = 0;
            WriteValue valueToWrite = new WriteValue();
            valueToWrite.NodeId = "ns=3;s=extraction_end";
            valueToWrite.AttributeId = Attributes.Value;
            valueToWrite.Value.Value = extraction_end;
            valueToWrite.Value.StatusCode = StatusCodes.Good;
            valueToWrite.Value.ServerTimestamp = DateTime.MinValue;
            valueToWrite.Value.SourceTimestamp = DateTime.MinValue;
            WriteValueCollection valuesToWrite = new WriteValueCollection();
            valuesToWrite.Add(valueToWrite);
            // write current value.
            StatusCodeCollection results = null;
            DiagnosticInfoCollection diagnosticInfos = null;
            m_session2.Write(
                null,
                valuesToWrite,
                out results,
                out diagnosticInfos);
            ClientBase.ValidateResponse(results, valuesToWrite);
            ClientBase.ValidateDiagnosticInfos(diagnosticInfos, valuesToWrite);
            if (StatusCode.IsBad(results[0]))
            {
                throw new ServiceResultException(results[0]);
            }
        }
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException("Error Writing Value", exception);
    }
}

```

Figura 19 - Função "FimExtracaoPeca_OFF()

4. MES

O *Manufacturing Execution System* (MES) do ManufacturerY comunica com a máquina de injeção de modo a enviar comandos de produção de peças. Neste trabalho foi criado um cliente OPC-UA “MES” para fazer a comunicação com o servidor da máquina e criada uma interface para ativar o envio dos pedidos de produção de forma manual.

Entre os pedidos que podem ser efetuados encontra-se o comando “start” para pedir que comece o processo de injeção e é também atribuído um número de série à peça. O cliente informa também qual a etapa atual do processo através da interface. Na Figura 20 é possível visualizar o aspeto da interface gráfica do “MESClient”.

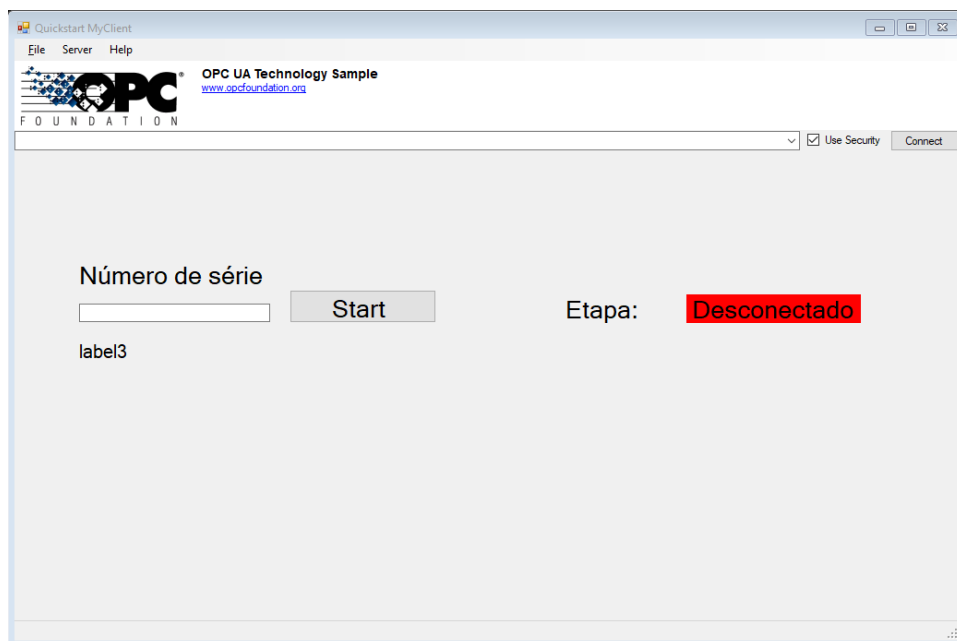


Figura 20 - Interface gráfica MES Client

Para configuração deste cliente, tal como para o “Extraction”, foi disponibilizada a localização do servidor a conectar associado ao “ConnectServerCTRL”, como pode ser visto na Figura 21.

```

public MainForm(ApplicationConfiguration configuration)
{
    InitializeComponent();
    ConnectServerCTRL.Configuration = m_configuration = configuration;
    ConnectServerCTRL.ServerUrl = "opc.tcp://localhost:62543/Quickstarts/MyModbus20pcuaServer";

    this.Text = m_configuration.ApplicationName;
}

```

Figura 21 - Endereço do servidor

Nos *Private Fields*, foi necessário declarar as respectivas variáveis a serem utilizadas. Na Figura 22 é possível visualizar estas declarações.

```

#region Private Fields
private ApplicationConfiguration m_configuration;
private Session m_session;
private bool m_connectedOnce;
private Subscription m_subscription;

//Variáveis IMM_Control Server
private double etapa;
private uint etapa_handle;

```

Figura 22 - Declaração de variáveis

Foi realizada a subscrição da variável “etapa” no nó 3 do servidor “IMMControl” como se pode ver na Figura 23.

```

//----- Server_ConnectComplete -----
1 reference
private void Server_ConnectComplete(object sender, EventArgs e)
{
    try
    {
        m_session = ConnectServerCTRL.Session;

        // set a suitable initial state.
        if (m_session != null && !m_connectedOnce)
        {
            m_connectedOnce = true;
        }

        //verify if Session is established
        if (m_session != null)
        {
            //verify if the Subscription was not already created
            if (m_subscription == null)
            {
                //create the subscription
                m_subscription = new Subscription(m_session.DefaultSubscription);
                m_subscription.PublishingEnabled = true;
                m_subscription.PublishingInterval = 4000; //msec interval
                m_session.AddSubscription(m_subscription);
                m_subscription.Create();

                MonitoredItem monitoredItem;
                //Add AtoB to monitored items
                monitoredItem = new MonitoredItem(m_subscription.DefaultItem);
                monitoredItem.StartNodeId = "ns=3;s=etapa";
                monitoredItem.AttributeId = Attributes.Value;
                monitoredItem.MonitoringMode = MonitoringMode.Reporting;
                monitoredItem.SamplingInterval = 1000;
                monitoredItem.QueueSize = 0;
                monitoredItem.DiscardOldest = true;
                //get handle number
                etapa_handle = monitoredItem.ClientHandle;
                //define event handler for this item, and then add to subscription
                monitoredItem.Notification += new MonitoredItemNotificationEventHandler(monitoredItem_Notification);
                m_subscription.AddItem(monitoredItem);
                m_subscription.ApplyChanges();
            }
        }

        // browse the instances in the server.
        //BrowseCTRL.Initialize(m_session, ObjectIds.ObjectsFolder, ReferenceTypeIds.Organizes, ReferenceTypeIds.Aggregates);
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException(this.Text, exception);
    }
}

```

Figura 23 - Subscrição da variável “etapa”

Na função “monitoredItem_Notification” foi criada uma condição para que seja lido o valor da variável “etapa” do servidor “IMMControl” e seja apresentado na interface do cliente, como se pode ver na Figura 24.

```

//----- IMM Control Server -----
if (monitoredItem.ClientHandle == etapa_handle)
{
    etapa = (double)notification.Value.WrappedValue.Value;

    label4.Text = Utils.Format("{0}", etapa.ToString());...
}

```

Figura 24 – Condição para apresentar valor de “etapa”

Na interface foi adicionado também um botão para que fosse possível dar o “start”. Dentro da função “button1_Click” foi adicionado uma condição para que apenas seja possível iniciar o processo se for inserido um número de série para a peça, caso contrário informa que o molde é inválido. Sendo

atribuído um número de série e carregador no botão, o valor 1 é atribuído à variável “start” no nosso servidor. Podemos encontrar este código na Figura 25.

```
//Botão start
1 reference
private void button1_Click(object sender, EventArgs e)
{
    try
    {
        if (m_session != null)
        {
            if (textBox1.TextLength == 0)
            {
                label3.Text = "Molde Inválido";
                label3.BackColor = Color.Red;
                label3.Visible = true;
            }
            else
            {
                label3.Visible = false;

                Int32 start = (Int32)1;
                WriteValue valueToWrite = new WriteValue();
                valueToWrite.NodeId = "ns=3;s=start";
                valueToWrite.AttributeId = Attributes.Value;
                valueToWrite.Value.Value = start;
                valueToWrite.Value.StatusCode = StatusCodes.Good;
                valueToWrite.Value.ServerTimestamp = DateTime.MinValue;
                valueToWrite.Value.SourceTimestamp = DateTime.MinValue;
                WriteValueCollection valuesToWrite = new WriteValueCollection();
                valuesToWrite.Add(valueToWrite);
                // write current value.
                StatusCodeCollection results = null;
                DiagnosticInfoCollection diagnosticInfos = null;
                m_session.Write(
                    null,
                    valuesToWrite,
                    out results,
                    out diagnosticInfos);
                ClientBase.ValidateResponse(results, valuesToWrite);
                ClientBase.ValidateDiagnosticInfos(diagnosticInfos, valuesToWrite);
                if (StatusCode.IsBad(results[0]))
                {
                    throw new ServiceResultException(results[0]);
                }
            }
        }
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException("Error Writing Value", exception);
    }

    nserie_Changed();
}
```

Figura 25 - Lógica da função “button1_Click”

No fim da função “button1_Click” encontramos chamada a função “nserie_Changed”. Esta função, como vemos na Figura 26, envia o número de série que foi escrito na interface para o servidor principal “IMMControl”.

```

//Texto n_serie
1 reference
private void nserie_Changed()
{
    try
    {
        if (m_session != null)
        {
            label3.Visible = false;
            string n_serie = textBox1.Text;
            WriteValue valueToWrite = new WriteValue();
            valueToWrite.NodeId = "ns=3;s=n_serie";
            valueToWrite.AttributeId = Attributes.Value;
            valueToWrite.Value.Value = n_serie;
            valueToWrite.Value.StatusCode = StatusCodes.Good;
            valueToWrite.Value.ServerTimestamp = DateTime.MinValue;
            valueToWrite.Value.SourceTimestamp = DateTime.MinValue;
            WriteValueCollection valuesToWrite = new WriteValueCollection();
            valuesToWrite.Add(valueToWrite);
            // write current value.
            StatusCodeCollection results = null;
            DiagnosticInfoCollection diagnosticInfos = null;
            m_session.Write(
                null,
                valuesToWrite,
                out results,
                out diagnosticInfos);
            ClientBase.ValidateResponse(results, valuesToWrite);
            ClientBase.ValidateDiagnosticInfos(diagnosticInfos, valuesToWrite);
            if (StatusCode.IsBad(results[0]))
            {
                throw new ServiceResultException(results[0]);
            }
        }
    }
    catch (Exception exception)
    {
        ClientUtils.HandleException("Error Writing Value", exception);
    }
}

```

Figura 26 - Função "nserie_Changed()"

5. IMM Energy Monitor e X Monitor

O servidor OPC-UA “IMM Energy Monitor”, localizado em autômato da Pheonix Contact, é responsável por disponibilizar informações relacionadas ao consumo de energia elétrica da máquina de injeção. O cliente “X Monitor” permite a visualização e a monitorização destas informações em tempo real, através da plataforma Node-RED.

Para a obtenção destes dados, foi necessário criar um conjunto de variáveis no servidor OPC-UA “IMM Energy Monitor”, utilizado para o cálculo da tensão e da corrente, conforme indica a tabela seguinte. Apenas as variáveis “rVoltage_V1RMS_V” e “rCurrent_A” estão declaradas neste servidor.

Nome da variável	Tipo de variável	Tipo de Uso	OPC UA
rActivePower_P_kW	REAL	Local	False
rVoltage_V1RMS_V	REAL	Local	True
RTC	RTC_TYPE	External	False
rActivePower_P_W	REAL	Local	False
rCurrent_A	REAL	Local	True

Os cálculos realizados para a obtenção dos dados da tensão e da corrente foram programados diretamente no autômato da seguinte maneira, ilustrada na figura 26.

```
//simulate values for the variables. Add some variability based on the system clock
rActivePower_P_kW := 5 + (RTC.SECONDS-30)*0.15;
rVoltage_V1RMS_V := 230 + (30-RTC.SECONDS)*0.5;

// Convert Active Power from kW to W (1 kW = 1000 W)
rActivePower_P_W := rActivePower_P_kW * 1000;

IF rVoltage_V1RMS_V > 0 THEN
    rCurrent_A := rActivePower_P_W / rVoltage_V1RMS_V;
ELSE
    rCurrent_A := 0;
END_IF;
```

Figura 27 – Cálculos da tensão e da corrente

A criação do cliente OPC-UA “X Monitor” é realizada na plataforma Node-RED, utilizando o *endpoint* gerado pelo servidor. Uma vez criado, é possível aceder aos dados da corrente e da tensão através do *address space* das variáveis disponibilizadas pelo servidor. Para a configuração da *dashboard* no Node-RED, foi necessário segmentar os dados do cliente OPC-UA com a ajuda do nó

“switch” (que utiliza o *address space* das variáveis). Após a separação, foi possível criar “gauges” e “chart” para mostrar os valores da corrente e da tensão. A figura 27 mostra o esquema descrito.

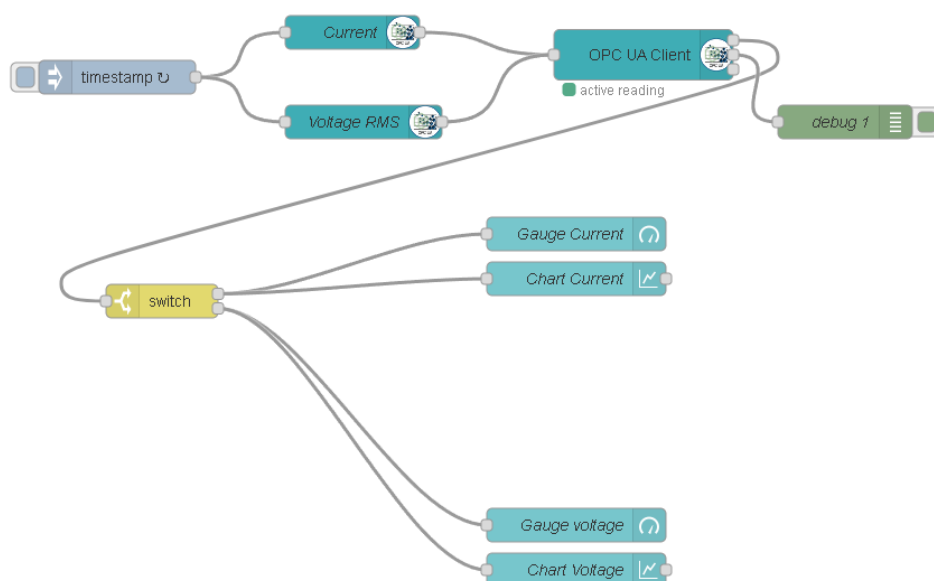


Figura 28- Esquema de acesso aos dados do servidor e de criação da dashboard

Após estabelecer a conexão entre servidor e cliente, foi possível visualizar a atualização simultânea dos valores da tensão e da corrente no Node-RED, juntamente com os dados do autômato da Pheonix Contact, permitindo a monitorização em tempo real.

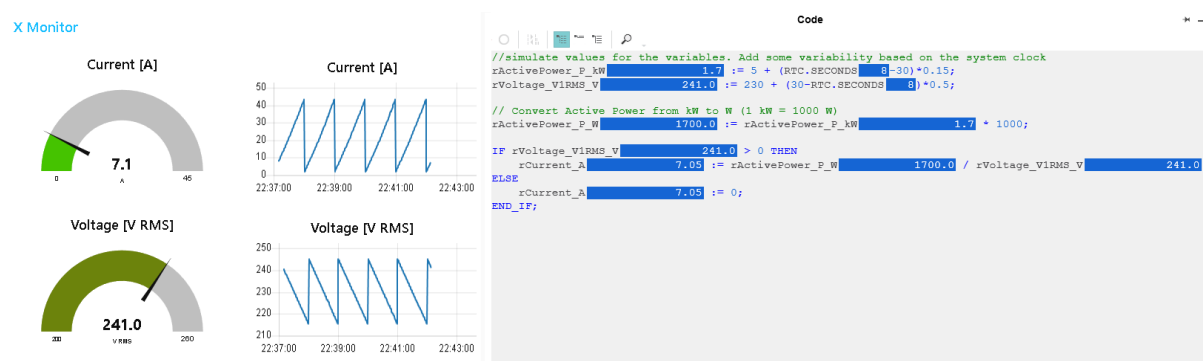


Figura 29 - Visualização e atualização dos valores da corrente e da tensão em tempo real

6. Conclusão

Com este trabalho pretendia-se explorar a criação de servidores e clientes OPC-UA que são bastante utilizados no âmbito da indústria 4.0. Estes clientes e servidores tinham funções específicas e variadas entre si, o que permitia explorar diferentes aplicações. Além da troca de informação entre servidores e clientes no Visual Studio, também havia informação proveniente de outros *softwares*, como o TIA Portal, Robot Studio, PLCNext da Phoenix Contact e plataforma Node-RED, o que nos permitiu desenvolver conhecimentos que podemos aplicar num caso real de uma indústria e novas competências e experiências.

Durante a realização do trabalho, foi necessário aprofundar conhecimentos em alguns conceitos para ser possível concluir os desafios propostos pois surgiram algumas dificuldades em algumas tarefas. Concluindo, ficámos satisfeitos com o trabalho realizado uma vez que nos permitiu adquirir alguns conhecimentos na área e aprendeu-se também que é possível adaptar um processo já existente ou mais antigo e colocá-lo moderno e compatível com a indústria 4.0, sem grandes problemas.